

Understanding Agentic AI Orchestration Frameworks

Learning How Modern AI Systems Work: LLMs, RAG, and Orchestration

Speedykom Group GmbH

February 25, 2026

Learning Objectives

At the end of this notebook, you will be able to:

- **Understand the building blocks of agentic AI:** LLMs, RAG (Retrieval Augmented Generation), orchestration
 - **Grasp orchestration frameworks:** How they coordinate multiple components (sensing, thinking, acting)
 - **Understand scaffolding and harnesses:** The patterns that make agents reliable and testable
 - **Compare existing tools:** OpenCode, Claude Code, Anticravity, OpenClaw, and others
 - **Explain why most frameworks use TypeScript:** And how Python fits into the ecosystem
 - **Build simple agents with Codetrain:** Understanding the mechanics before production
 - **Know the difference:** CodeTrain (educational) vs. production frameworks (feature-rich)
 - **Design systems that scale:** From prototype to understanding production architectures
-

Part 0: The Building Blocks - Before We Build Anything

Foundation: What Is an LLM?

An **LLM (Large Language Model)** is a machine learning model trained on billions of words to predict the next word in a sequence.

What it does:

Input: "The capital of France is"
LLM predicts: "Paris"

Input: "Write a Python function to sort a list"
LLM predicts: "def sort_list(items): return sorted(items)"

Key insight: LLMs don't "understand" - they pattern match at scale. But this is powerful because language contains knowledge.

Examples of LLMs:

- GPT-4 (OpenAI) - Most capable
 - Claude 3.5 (Anthropic) - Good at reasoning
 - Llama 3.1 (Meta) - Open source, free
 - Gemini (Google) - Multimodal
-

Foundation: What Is RAG (Retrieval Augmented Generation)?

The problem: LLMs have a knowledge cutoff (they don't know current events) and can hallucinate (make up facts).

The solution: RAG = "Retrieve documents, then ask LLM to answer based on those documents"

User Question: "What are the features of Codetrain?"

↓

RETRIEVE: Search for documents about Codetrain

↓

LLM: "Based on these documents, Codetrain is..."

↓

Answer: Grounded in actual information, not hallucination

Real-world example:

Question: "What's on the CodeTrain Africa website?"

Without RAG:

- LLM: "I don't know, my training data is from before CodeTrain existed"

With RAG:

- Crawler: Fetches CodeTrain website
- Retriever: Finds relevant sections
- LLM: "Based on the website, CodeTrain Africa offers..."

Components of RAG:

1. **Crawler/Retriever** - Gets relevant documents
 2. **Vector Database** - Stores embeddings for semantic search
 3. **Ranker** - Decides which documents are most relevant
 4. **LLM** - Generates answer based on documents
-

Foundation: What Is Orchestration?

Orchestration = Coordinating multiple steps in sequence, with logic to decide what happens next.

Simple example:

Step 1: User asks a question

Step 2: System decides: "Do I need to search the web?"

YES → Search the web

NO → Use my training knowledge

Step 3: System decides: "Do I need to use a tool?"

YES → Call calculator, database, API

NO → Just answer

Step 4: Generate response

Step 5: Format and return to user

Why orchestration matters:

- LLMs are good at thinking but not at *doing*
 - An agent needs to: sense → decide → act → repeat
 - Orchestration is the glue that makes this work
-

Foundation: Scaffolding & Harnesses

Scaffolding = Structure and patterns that make agents reliable

Harnesses = Testing frameworks that verify agents work correctly

Example of bad agent (no harness):

```
# This might work, might not
def my_agent(question):
    answer = llm.ask(question)
    return answer
```

Example of good agent (with harness):

```
def my_agent(question):
    # Validate input
    if not question or len(question) < 5:
        return {"error": "Question too short"}

    # Execute with retry logic
    for attempt in range(3):
        try:
            answer = llm.ask(question)
            break
        except APIError:
            if attempt == 2:
                return {"error": "API failed after 3 attempts"}
            wait(2 ** attempt) # exponential backoff

    # Validate output
    if len(answer) == 0:
        return {"error": "LLM returned empty response"}

    return {"success": True, "answer": answer}

# Harness: Test cases
test_cases = [
    ("short", False), # Should reject
    ("What is 2+2?", True), # Should answer
    ("", False), # Should reject empty
]

for question, should_succeed in test_cases:
    result = my_agent(question)
    assert (result.get("success", False) == should_succeed)
```

Part 1: What Is CodeTrain and Why It Was Created?

The History: From PocketFlow to CodeTrain

PocketFlow (created ~2020-2021):

- Early orchestration framework for agents
- Built by Speedykom for internal AI systems

- Worked well for specific use cases

Why create CodeTrain?

- PocketFlow was closed-source and internal-only
- Market was moving toward agent frameworks
- Need for an **open, educational framework** to teach students how orchestration actually works
- Most frameworks (Langchain, CrewAI, etc.) are complex and production-focused

CodeTrain's purpose:

- Take PocketFlow's core ideas
 - Simplify for educational use
 - **Show students how agents work without production complexity**
 - Framework that's:
 - Simple enough to understand
 - Complete enough to be real
 - Educational, not production
-

What Makes CodeTrain Different

CodeTrain = Educational Orchestration Framework

Core idea: "Show students the minimum viable structure for an agent"

Traditional Framework (Langchain, CrewAI):

50+ files
 Complex state management
 Production features (caching, logging, metrics)
 Steep learning curve

CodeTrain:

~10 files
 Clear job/task separation
 Focus on "what's necessary to learn?"
 Fast to understand

Philosophy: - "Learn by building" - Not "here's a black box, use it" - See every step of the orchestration

Part 2: The Orchestration Landscape - Tools and Tradeoffs

Existing Orchestration Frameworks

1. OpenCode (Open source agent CLI)

- **Built with:** TypeScript/JavaScript
- **Runs:** In your terminal
- **Best for:** Developer tools, CLI automation
- **Example:** File operations, code generation, git workflows
- **Why TypeScript:** Fast execution, good JavaScript ecosystem

2. Claude Code (Anthropic's IDE)

- **Built with:** TypeScript (frontend), runs Claude 3.5 backend
- **Runs:** In Claude.ai or IDE extensions

- **Best for:** Pair programming, real-time code assistance
- **Example:** Build features, debug code, refactor
- **Why TypeScript:** Browser compatibility, modern tooling

3. Anticravity (No-code agent platform)

- **Built with:** TypeScript/JavaScript (frontend), Python (backend)
- **Runs:** Web-based
- **Best for:** Non-technical users, business workflows
- **Example:** Sales automation, customer service, data pipelines
- **Why TypeScript:** Web UI, real-time updates

4. OpenClaw (Open source framework)

- **Built with:** TypeScript
- **Runs:** Node.js
- **Best for:** JavaScript-heavy applications
- **Example:** Web scraping, API automation
- **Why TypeScript:** Native async/await, npm ecosystem

5. CodeTrain (Educational framework)

- **Built with:** Python
- **Runs:** Anywhere Python runs
- **Best for:** Learning, teaching, understanding orchestration
- **Example:** Web crawlers, chatbots, research tools
- **Why Python:** Data science integration, Jupyter notebooks

6. Langchain (Popular framework)

- **Built with:** Python (main), TypeScript bindings
- **Runs:** Anywhere
- **Best for:** Production LLM applications
- **Complexity:** High (1000+ files)
- **Cost:** High (many abstractions = slower)

Why Most Frameworks Use TypeScript

The TypeScript Advantage

1. **Web first:** JavaScript runs in browsers
2. **Real-time UI:** WebSockets, live updates
3. **Performance:** Fast compilation and execution
4. **Ecosystem:** npm has everything (axios, puppeteer, express, etc.)
5. **Modern async:** async/await is first-class

But Python Has Advantages Too

1. **Data science:** NumPy, Pandas, scikit-learn integration
 2. **ML models:** PyTorch, TensorFlow
 3. **Readability:** Cleaner syntax
 4. **Learning curve:** Easier for beginners
 5. **Scientific tools:** Jupyter, notebooks
-

The Hybrid Solution: Python + TypeScript

Smart systems combine both:

TypeScript (Frontend):

- Web UI, real-time updates, user interactions

Python (Backend):

- Data processing with Pandas
- ML models with scikit-learn
- Orchestration with CodeTrain
- APIs with FastAPI

Communication:

- HTTP/WebSocket between them

Why this works: - JavaScript for what it's good at (UIs, real-time) - Python for what it's good at (data, ML, orchestration) - Best of both worlds

Part 3: Installing CodeTrain

Setup Instructions

Using uv (Recommended)

```
uv pip install git+https://github.com/Speedykom/CodeTrain.git
```

Using pip

If you don't have uv installed:

```
pip install git+https://github.com/Speedykom/CodeTrain.git
```

Virtual Environment (Highly Recommended)

Always install in a virtual environment to avoid conflicts:

```
# Using uv (recommended)
uv venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate
uv pip install git+https://github.com/Speedykom/CodeTrain.git

# Alternative: Using standard Python
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install git+https://github.com/Speedykom/CodeTrain.git
```

Verify Installation

```
import codetrain as ct
print(ct.__version__)
```

Part 4: Understanding the Three-Stage Job Pattern

CodeTrain is built on the **logistics pattern**: every unit of work has three stages.

The Three Stages

Every **Job** in CodeTrain has three mandatory methods:

1. `receive_order(manifest)` — Input Stage

- **Purpose**: Extract and validate input from the shared manifest
- **Returns**: Data to pass to `prepare_order`
- **Analogy**: Receiving a delivery order, checking what's needed

2. `prepare_order(data)` — Work Stage

- **Purpose**: Do the actual work (call LLM, process data, etc.)
- **Returns**: The result of the work
- **Analogy**: Packaging and preparing the item for delivery

3. `ship_order(manifest, input_data, result)` — Output Stage

- **Purpose**: Store the result back in the manifest
- **Returns**: An action name for conditional routing (or “continue” for loops)
- **Analogy**: Handing off to delivery and updating tracking info

Simple Example: A Greeting Job

```
import codetrain as ct

class GreetingJob(ct.Job):
    """A simple job that greets someone."""

    def receive_order(self, manifest):
        """Load data from the manifest."""
        name = manifest.get("name", "World")
        return name

    def prepare_order(self, name):
        """Do the actual work."""
        greeting = f"Hello, {name}!"
        return greeting

    def ship_order(self, manifest, name, greeting):
        """Store the result back in the manifest."""
        manifest["greeting"] = greeting
        manifest["processed_at"] = "just now"
        return "done"

# Create and run the job
job = GreetingJob()
manifest = {"name": "Ghana"}
job.run(manifest)

print(manifest["greeting"]) # Output: Hello, Ghana!
```

Connecting Jobs: The » Operator

Real workflows need multiple jobs. CodeTrain uses >> to chain them:

```
import codetrain as ct

class ExtractNameJob(ct.Job):
    def receive_order(self, manifest):
        return manifest.get("user_input", "")

    def prepare_order(self, user_input):
        return user_input.strip()

    def ship_order(self, manifest, user_input, name):
        manifest["name"] = name
        return "done"

class ValidateNameJob(ct.Job):
    def receive_order(self, manifest):
        return manifest.get("name", "")

    def prepare_order(self, name):
        if len(name) < 2:
            raise ValueError("Name must be at least 2 characters")
        return name.upper()

    def ship_order(self, manifest, name, validated_name):
        manifest["validated_name"] = validated_name
        manifest["is_valid"] = True
        return "done"

# Chain jobs using >>
extract = ExtractNameJob()
validate = ValidateNameJob()

workflow = extract >> validate

# Run with Hustle
hustle = ct.Hustle(start=workflow)
manifest = {"user_input": " kwame "}
hustle.run(manifest)

print(manifest["validated_name"]) # Output: KWAME
```

Read the » operator as “then”: - job_a >> job_b means “do job_a, **then** pass manifest to job_b” - You can chain as many as needed: a >> b >> c >> d >> e

Conditional Routing: The - "action" >> Syntax

Sometimes different inputs need different paths. Use conditional routing:


```

class CheckAgeJob(ct.Job):
    """Checks if user is an adult."""

    def receive_order(self, manifest):
        return manifest.get("age", 0)

    def prepare_order(self, age):
        return age >= 18

    def ship_order(self, manifest, age, is_adult):
        manifest["is_adult"] = is_adult
        # Return different action based on result
        return "adult" if is_adult else "minor"

class AdultPathJob(ct.Job):
    def receive_order(self, manifest):
        return manifest.get("name", "User")

    def prepare_order(self, name):
        return f"Welcome to adult section, {name}!"

    def ship_order(self, manifest, name, message):
        manifest["content"] = message
        return "done"

class MinorPathJob(ct.Job):
    def receive_order(self, manifest):
        return manifest.get("name", "User")

    def prepare_order(self, name):
        return f"Welcome to kids section, {name}! "

    def ship_order(self, manifest, name, message):
        manifest["content"] = message
        return "done"

# Set up conditional routing
check = CheckAgeJob()
adult_job = AdultPathJob()
minor_job = MinorPathJob()

check - "adult" >> adult_job
check - "minor" >> minor_job

# Test
hustle = ct.Hustle(start=check)
manifest = {"name": "Kwame", "age": 25}
hustle.run(manifest)
print(manifest["content"]) # "Welcome to adult section, Kwame!"

```

Part 5: Building Your First Agent with CodeTrain

The Architecture

CodeTrain agents work in **jobs** and **stages**:

A Job = A discrete piece of work

Job Example 1: CrawlJob

Input: URL

Action: Fetch page, extract links and text

Output: Page content + found links

Job Example 2: DecideJob

Input: Question + crawled content

Action: Ask LLM "do we have enough info?"

Output: "answer" or "explore more"

Job Example 3: AnswerJob

Input: Question + relevant content

Action: Synthesize answer from content

Output: Natural language answer

Example 1: Simple Chatbot Using CodeTrain

Goal: Build a chatbot that answers questions about a website

```
from codetrain import Job, Orchestrator
from codetrain.jobs import CrawlJob, DecideJob, AnswerJob

class ChatbotOrchestrator(Orchestrator):
    """
    Orchestration logic:
    1. Crawl the website
    2. Ask LLM: "Do I have enough info?"
    3. If yes → synthesize answer
    4. If no → crawl more pages, loop back
    """

    async def run(self, user_question: str):
        # Stage 1: Crawl initial page
        crawl_result = await CrawlJob(
            url="https://example.com",
            question=user_question
        ).execute()

        # Stage 2: Decide if we need more info
        decide_result = await DecideJob(
            content=crawl_result["text"],
            question=user_question
        ).execute()

        if decide_result["action"] == "answer":
            # We have enough info
            answer = await AnswerJob(
```

```

        content=crawl_result["text"],
        question=user_question
    ).execute()
    return answer["response"]

else:
    # Need to explore more
    for url in crawl_result["links"][:3]: # Try up to 3 more pages
        crawl_result = await CrawlJob(
            url=url,
            question=user_question
        ).execute()

        decide_result = await DecideJob(
            content=crawl_result["text"],
            question=user_question
        ).execute()

        if decide_result["action"] == "answer":
            answer = await AnswerJob(
                content=crawl_result["text"],
                question=user_question
            ).execute()
            return answer["response"]

    # If we exhausted pages, answer with what we have
    return "Could not find specific answer. Try rephrasing your question."

# Usage
orchestrator = ChatbotOrchestrator()
response = await orchestrator.run("What services do you offer?")
print(response)

```

Key insight: Notice how orchestration separates concerns: - Crawling logic in CrawlJob - Decision logic in DecideJob - Answer synthesis in AnswerJob - The orchestrator just coordinates them

Example 2: Building a Research Assistant

Goal: Analyze multiple sources and synthesize findings

```

class ResearchAssistant(Orchestrator):
    """
    Orchestration:
    1. Search for papers/articles
    2. Read each one
    3. Extract key findings
    4. Synthesize summary
    5. Identify gaps
    """

    async def run(self, research_topic: str):
        sources = await SearchJob(query=research_topic).execute()

```

```

findings = []
for source in sources["results"][:5]: # Top 5 sources
    content = await CrawlJob(url=source["url"]).execute()
    finding = await ExtractJob(
        content=content["text"],
        topic=research_topic
    ).execute()
    findings.append(finding)

summary = await SynthesisJob(
    findings=findings,
    topic=research_topic
).execute()

gaps = await GapAnalysisJob(
    summary=summary,
    topic=research_topic
).execute()

return {
    "summary": summary["text"],
    "gaps": gaps["identified_gaps"],
    "sources": sources
}

```

Part 4: CodeTrain vs. Production Frameworks

When to Use CodeTrain

Scenario	Use CodeTrain?
Learning how agents work	YES
Teaching students	YES
Building prototypes	YES
Understanding orchestration	YES
Quick experiments	YES

When NOT to Use CodeTrain

Scenario	Use CodeTrain?	Use Instead
Production systems serving 1000+ users	NO	Langchain, CrewAI
Enterprise with SLAs	NO	Purpose-built frameworks
Complex caching/logging needs	NO	Production frameworks
Scaling to 10K requests/second	NO	Cloud-native solutions

The Reality: Why Production Frameworks Exist

Production frameworks (Langchain, CrewAI) have:

- Advanced caching
- Distributed execution
- Monitoring and logging
- Error recovery
- Rate limiting
- Cost optimization
- Security features
- Database persistence
- Multi-tenant support
- Scaling strategies

CodeTrain has:

- Core orchestration logic
- Simplicity
- Educational clarity
- Rapid prototyping

Analogy:

- CodeTrain = Learning to drive (simple car)
- Production frameworks = Professional delivery services (complex logistics)

The path:

1. Learn with CodeTrain → Understand how orchestration works
 2. Prototype with CodeTrain → Build your idea
 3. Move to production framework → Deploy at scale
 4. Use production framework's advanced features → Optimize costs, reliability
-

Part 5: How Python Fits Into the AI Stack

The Evolution of AI Development

Era 1: Python-First (2015-2018)

- All AI work in Python
- Web frontends separate (JavaScript)
- Not designed for real-time interaction

Era 2: JavaScript Takes Over (2018-2020)

- JavaScript AI libraries (TensorFlow.js)
- Frameworks in TypeScript (fast, type-safe)
- Real-time, web-native

Era 3: Hybrid (2020-present)

- Python for heavy lifting (data, ML)
- JavaScript/TypeScript for UI
- They communicate via APIs

Modern stack:

User Interface (JavaScript/React)
↑ HTTP/WebSocket ↓
Fast API Server (Python)
Data processing (Pandas)
ML models (scikit-learn)
Orchestration (CodeTrain)
LLM APIs

Where Python Excels

1. Data Science Pipeline

```
import pandas as pd

# Load, clean, analyze customer data
customers = pd.read_csv('customers.csv')
high_value = customers[customers['lifetime_value'] > 10000]
patterns = high_value.groupby('region').mean()
```

2. ML Model Integration

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

3. Orchestration

```
from codetrain import Orchestrator

# Coordinate multiple steps
result = await orchestrator.run(query)
```

Part 6: Real Examples - What You Can Build

Example 1: Self-Looping Interactive Chatbot

Use case: Build an interactive chatbot that maintains conversation history

Key Concept: Self-Looping Pattern

A job can loop back to itself using - "action" >> syntax:

```
chat_job - "continue" >> chat_job
```

This creates an infinite loop where the job keeps running until it returns `None` (end conversation).

Implementation:

```
import codetrain as ct
import random

class InteractiveChatJob(ct.Job):
    """A chatbot that loops and maintains conversation history."""
```

```

def receive_order(self, manifest):
    """Get user input, initialize conversation history."""
    # First run: set up
    if "messages" not in manifest:
        manifest["messages"] = []
        print("=" * 60)
        print(" CodeTrain Interactive Chatbot")
        print("=" * 60)
        print("Type 'exit' to end\n")

    # Get user input
    user_input = input("You: ").strip()

    # Check for exit
    if user_input.lower() == "exit":
        return None

    # Store in history
    manifest["messages"].append({"role": "user", "content": user_input})
    return user_input

def prepare_order(self, user_input):
    """Generate response based on input."""
    if user_input is None:
        return None

    # Keyword-based responses
    responses = {
        "hello": "Hello! How can I help?",
        "codetrain": "CodeTrain is a minimal orchestration framework!",
        "job": "A Job has three stages: receive → prepare → ship",
        "help": "I can talk about CodeTrain concepts!",
    }

    # Check keywords
    for keyword, response in responses.items():
        if keyword in user_input.lower():
            return response

    # Default
    return random.choice([
        "That's interesting!",
        "Tell me more!",
        "I see what you mean.",
    ])

def ship_order(self, manifest, user_input, response):
    """Store response and decide what to do next."""
    if user_input is None:
        print("\n" + "=" * 60)
        print(" Goodbye!")
        print("=" * 60)
        return None # End conversation

```

```

    # Print response
    print(f"Bot: {response}\n")

    # Store in history
    manifest["messages"].append({"role": "bot", "content": response})

    # Loop back for more input
    return "continue"

# Create and run
chat_job = InteractiveChatJob()
chat_job - "continue" >> chat_job # Self-loop

hustle = ct.Hustle(start=chat_job)
manifest = {}
hustle.run(manifest)

# After chat ends, you have full conversation history
print(f"Total messages: {len(manifest['messages'])}")

```

Why self-looping is powerful:

- Stateful conversation (manifest persists across loops)
- Simple logic (just return “continue” to keep going)
- Clear exit condition (return None to stop)
- Access to full history after completion

Learning value: Understand how agents maintain state and handle interactive workflows

Example 2: Educational Chatbot

Use case: Help students with programming questions

Architecture:

Student Question

↓

CodeTrain Orchestrator:

```

    Search documentation
    Check if answer is clear
    If not → Search related topics
    Synthesize explanation

```

Student-Friendly Response

Learning value: Understand how search + synthesis + validation works

Example 2: Research Aggregator

Use case: Analyze multiple papers on a topic

Architecture:

Research Topic

↓
CodeTrain Orchestrator:
Find papers
Crawl each paper
Extract key claims
Find contradictions
Identify gaps

Structured Research Report

Learning value: Understand parallel processing, conflict resolution, synthesis

Example 3: Data Analysis Pipeline

Use case: Analyze customer data and generate insights

Architecture:

Raw Data
↓
CodeTrain + Pandas:
Load and clean
Identify patterns
Generate hypotheses
Test hypotheses
Synthesize insights

Executive Summary

Learning value: Understand how agents work with data science

Part 7: Key Takeaways

What You Should Understand Now

1. **LLMs** predict text based on patterns
2. **RAG** grounds predictions in real documents
3. **Orchestration** coordinates multiple steps with logic
4. **CodeTrain** is an educational framework that shows core concepts
5. **Production frameworks** add complexity for reliability/scale
6. **Python + TypeScript** hybrid is the modern approach
7. **CodeTrain is for learning**, not production

The Learning Journey

Week 1-2: Understand LLMs
↓
Week 3-4: Understand RAG
↓
Week 5-6: Build with CodeTrain (simple agents)
↓
Week 7-8: Compare with production frameworks
↓

Week 9-10: Understand when to use each

↓

Week 11-12: Build capstone project (your choice)

Final Insight: Why This Matters for Your Career

2024-2025 Reality:

- Every company is building AI systems
- They're looking for people who understand **orchestration**
- Most job postings say “Langchain” or “CrewAI” but what they mean is “understands how to coordinate AI components”
- **CodeTrain teaches you that core skill**

Once you know orchestration:

- You can pick up any framework in a weekend
 - You can architect complex systems
 - You can optimize for your company's needs
 - You'll be valuable in the AI economy
-

Status: Ready for workshop. Students will build real agents while understanding the mechanics.